



软件开发技术

第7章 面向对象软件开发技术基础

浙江大学 机械工程学院 航空制造工程研究所

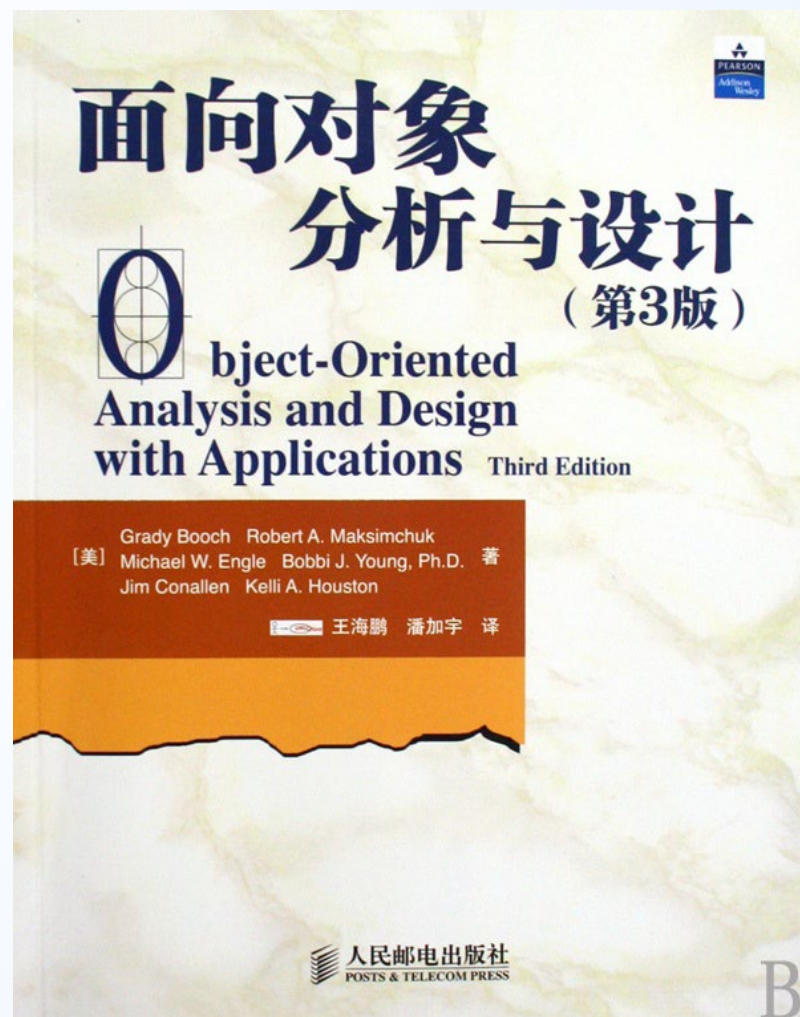
王青 李江雄

□ 1. 面向对象基本概念

- ✓ 1.1 对象模型基础
- ✓ 1.2 对象模型要素
- ✓ 1.3 应用对象模型

□ 2. 类与对象

- ✓ 2.1 理解类与对象
- ✓ 2.2 对象的本质
- ✓ 2.3 对象之间的关系
- ✓ 2.4 类的本质
- ✓ 2.5 类之间的关系
- ✓ 2.6 创建高品质的类与对象





01

面向对象的基本概念

面向对象建模与设计是**使用真实世界的概念模型来思考问题**的一种方法。面向对象建模与设计过程最终得到的是**“对象”及“对象”之间的关系（表达真实世界的模型）**，“对象”既包含表达真实世界某个或某类实体的数据结构，又包含对该实体的行为抽象。

软件开发者利用面向对象的模型，表达了对用户需求（领域知识）的理解，并将它作为与用户（领域专家）讨论的基础。

软件开发的各个阶段、软件开发项目组成员间，通过该模型来传达设计思想。

该模型是软件实现的基础，软件实现的过程，就成为“用一个给定的编程语言实现该模型”的过程。

面向对象的软件开发中，在软件生命周期各个阶段都使用同一套模型，而不像结构化开发方法中，在不同的阶段使用不同模型（图）来表达开发结果。

结构化的设计方法指导开发者利用**算法**作为基本构建块来构建复杂系统，面向对象设计方法则利用**类和对象**作为基本构建块。

对象模型不只是面向对象程序设计中使用，它已被证实是计算机科学中的一个统一概念。Yonezawa和Tokoro指出：术语“对象”在计算机科学的领域中独立地出现，几乎都是在20世纪70年代初期，它们出现时指的是不同的概念，但互相有关。所有这些概念的发明都是为了**管理系统的复杂性**，即**对象代表了按模块分解的系统的组件**，或者是知识表达的模块化单元。Levy补充指出，下列事件促进了面向对象概念的演进：

- ❑ 计算机架构的进步，包括基于功能的系统以及对操作系统概念的硬件支持；
- ❑ 程序设计语言的进步；
- ❑ 编程方法学的进步，包括模块化和信息隐藏；
- ❑ 数据库模型的进步，主要是通过E-R图的方式进行数据建模；
- ❑ 对人工智能的研究，知识表示的进步对理解对象的抽象做出了贡献；
- ❑ 哲学和认知科学领域的进步，笛卡尔说人们自然的用面向对象观点看世界。

面向对象分析和设计代表了一种**演进式开发**，而不是一种革命式开发，它没有抛弃过去的优点，而是建立在已经证明的好方法的基础之上。许多优秀的工程师利用结构化设计技术开发并部署了无数有用的软件系统，然而只利用算法分解在处理大量复杂性的情况时是有局限的，所以我们必须转向面向对象分解。而且如果试图将C++和Java这样的语言当作传统的面向算法语言来使用就丧失了语言提供的威力，甚至不如传统的C和Pascal语言。**就如一个木匠不懂电，给他一个电钻也只能当锤子使用。**

对象是什么？对象在真实世界中存在吗？

- ◆ 对象是一个种类的实例，如你、我都是人这个种类的一个实例。在面向对象中，**种类就用类来表示**。对象具有自身的结构，该结构包含了属性和行为，所谓对象的行为是指其所能执行的操作。**属性和操作合起来就称为对象的特征**。
- ◆ 对象，不论是具体的还是抽象的，遍布于我们的周围，它们组成了整个世界。**软件是用来模拟真实世界的某个部分的**，因此，用程序来模拟真实世界中的对象，对于用软件来解决真实世界中的问题是相对自然的。
- ◆ **属于同一个类的对象，就要一些共同的属性**，如你、我都属于人这个类，我们具有共同的属性：如年龄、体重、身高、学历等。同时，我们都能执行一些共同的操作：如说话、吃饭、读书、写字。
- ◆ 在软件中，**类是用来创建对象的模板**，每实例化一次类，就得到一个对象，该对象具有类的所有属性并能执行类的所有操作，**对象的属性有具体的值—属性值**。
- ◆ 由于，软件只是模拟真实世界的一部分，所以**同一事物在不同的应用领域只抽象出其所有属性中与应用领域相关的数据和操作**。

◆对象

“对象”是面向对象方法中的最基本的概念。

对象是系统中用来描述客观事物的一个实体，它是构成系统的一个基本单位。

一个对象由描述该对象的一组属性和对这组属性进行操作的一组方法组成。

对象可以是具体的物理实体的抽象，也可以是人为的概念，或者是任何有明确边界和意义的东西。

●对象的定义：

定义1（**从程序设计角度看对象**）：对象是具有相同状态的一组操作的集合；

定义2（**从信息模拟的角度看对象**）：对象是对问题域中某个东西的抽象，这种抽象反映了系统保存有关这个东西的信息或与它交互的能力，也就是说对象是属性值和操作的封装；

定义3（**对象的形式化定义**）：对象： $:\text{ = } \langle \text{ID}, \text{MS}, \text{DS}, \text{MI} \rangle$

ID：对象的标识

MS：对象的操作集合

DS：对象的数据结构

MI：对象受理消息名集合（对象的对外的接口）

定义4：对象代表了一个单独的、可确认的物体、单元或实体，它可以是具体的也可以是抽象的，它在问题领域有确切定义和角色。它具有状态、行为或标识。

1.1.1 面向对象编程（OOP）

定义：面向对象编程是一种**实现**方法，在这种方法中，程序被组织成许多组相互协作的对象，**每个对象代表某个类的一个实例，而类则属于一个通过继承关系形成的层次结构。**

这个定义有三个要点：

- (1) 利用对象作为面向对象编程的基本逻辑构建块，而不是利用算法；
- (2) 每个对象都是某个类的一个实例；
- (3) 类与类之间可以通过继承关系联系在一起。

1.1.2 面向对象设计（OOD）

编程方法中的重点是要正确有效地使用特定的语言机制，而设计方法的重点是正确有效地构造出复杂系统的结构。

定义：面向对象设计是一种**设计**方法，包括面向对象分解的过程和一种表示方法，这种表示法用于展现被设计系统的**逻辑模型**和**物理模型**、**静态模型**和**动态模型**。

这个定义有两个要点：

- (1) 面向对象设计导致了**面向对象分解**，面向对象分解的支持是面向对象设计与结构化设计的不同之处：前者利用**类和对象的抽象**来构建逻辑系统结构，后者则利用算法抽象；
- (2) 面向对象设计使用了不同的表示方法来表达系统逻辑设计（类和对象结构）和物理设计（模块和处理架构）的不同模型，以及系统的静态和动态特征。

1.1.3 面向对象分析（OOA）

传统的结构化分析技术关注系统中的**数据流**，而面向对象分析的重点在于利用面向对象的观点来看世界，**利用对象来构建真实世界的模型**。

定义：面向对象分析是一种**分析**方法，这种方法利用问题域的词汇表来**找到系统中候选的类和对象来分析需求**。

OOA、OOD和OOP之间的关系：面向对象分析的结果可以作为开始面向对象设计的模型，面向对象设计的结果可以作为蓝图，利用面向对象编程方法最终实现一个系统。

1.2 对象模型要素

面向对象模型有4个主要要素：

- ◆ (1) 抽象
- ◆ (2) 封装
- ◆ (3) 模块化
- ◆ (4) 层次结构

所谓“主要”，指的是如果一个模型不具备这些要素之一，就不是面向对象的。

面向对象模型有3个次要要素：

- ◆ (1) 类型
- ◆ (2) 并发
- ◆ (3) 持久

所谓“次要”，指的是这些要素是对象模型的有用组成部分，但不是本质的。

1.2.1 抽象的意义

抽象是人类处理复杂性的基本方式，以下是抽象的几种定义：

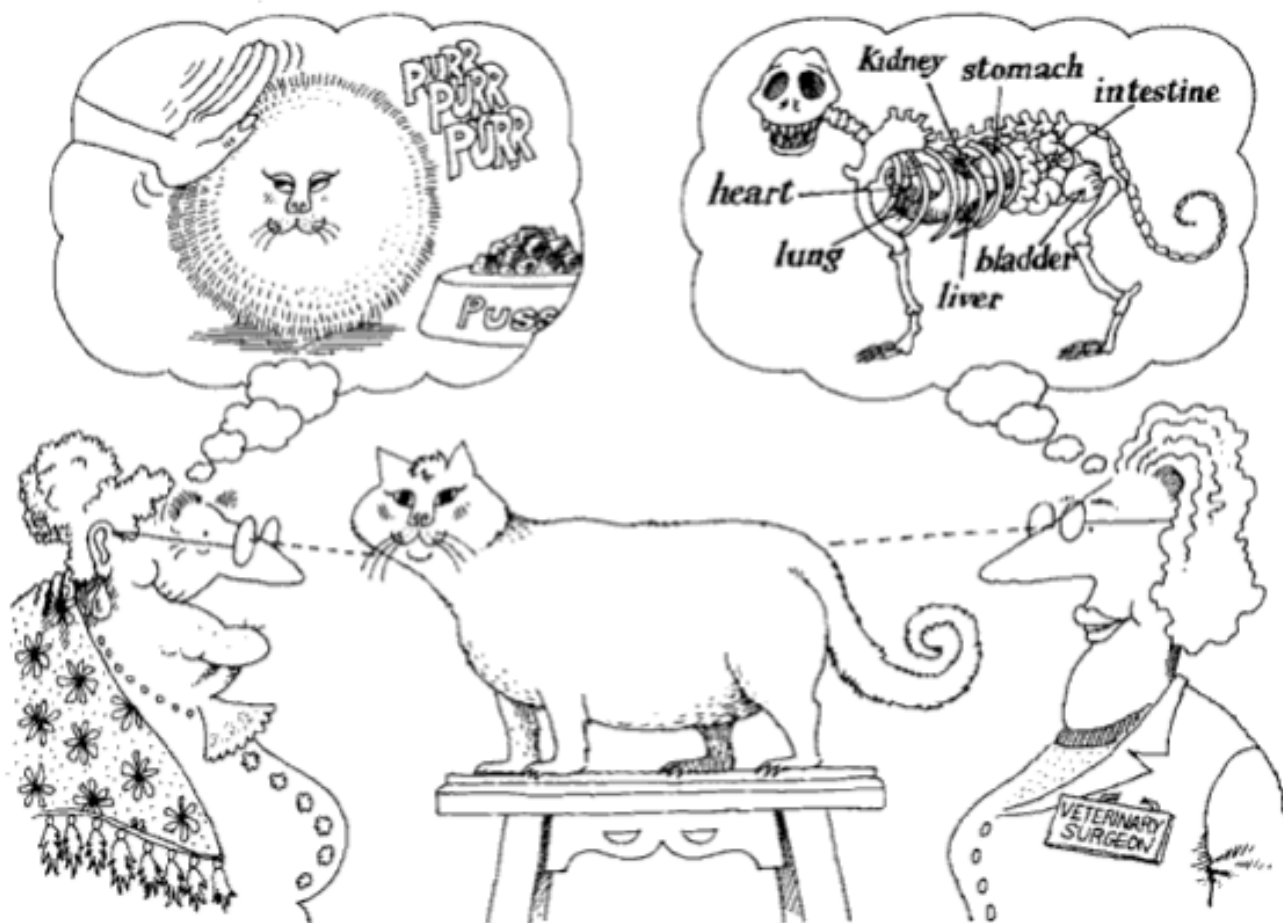
抽象是从众多的事物中抽取出共同的、本质性的特征，而舍弃其非本质的特征。例如苹果、香蕉、生梨、葡萄、桃子等，它们共同的特性就是水果。得出水果概念的过程，就是一个抽象的过程。（来自百度百科）

Grady Booch定义：抽象描述了一个对象的基本特征，可以将这个对象与所有其他类型的对象区分开来，因此提供了清晰定义的概念边界，但它与观察者的视角有关。

抽象关注一个对象的**外部视图**，所以可以用来分离对象的基本行为和它的实现，这种行为/实现的分界称为**抽象壁垒**，它通过“最小承诺”和“最小惊奇”原则来达成。

最小承诺原则：对象的接口只提供它的基本行为；

最小惊奇原则：捕捉某个对象的全部有用行为，不多也不少，并且不能提供抽象之外的惊奇效果或副作用。



抽象关注某个对象的基本特征，它与观察者的视角有关

抽象的例子：

在一个溶液栽培的种植园中，作物是在营养液中栽培的，没有沙、碎石或其他土壤，维护正常的温室环境是一件精细的工作，这取决于栽培作物的种类和它的生长阶段。人们必须控制各种因素，如温度、湿度、光照、PH值和营养液的浓度。

这个问题的关键抽象是**传感器**，实际上系统中存在几种不同类型的传感器，包括空气温度、水温、湿度、光照、PH值和溶液浓度的传感器和其他一些传感器。以下以温度传感器为例进行抽象。

在外部看来，温度传感器仅仅是一个对象，它知道如何测量某个具体位置的温度。

□ **什么是温度？** 它是一个数值，具有一定的范围和一定的精度，采用华氏、摄氏或开式温标。

□ **什么是位置？** 它是种植园中某个可标示的地方，我们希望测量那里的温度。对于温度传感器对象而言，重要的不是它处于什么位置，而是它有一个位置，并有一个唯一的标识符，用于区分它和其他所有传感器。

□ **温度传感器的责任是什么？** 传感器负责知道某个位置的温度，并在询问时报告温度。更确切的说是一个客户可以对温度传感器执行哪些操作？**定标，询问当前温度；**

目前为止，抽象都是被动的，温度传感器改为主动的，也就是它不需要其他对象来激活，而是当它所在位置的温度改变了一定值时，激活其他对象。

温度传感器的抽象：

```
//Temperature in degrees Fahrenheit
typedef float Temperature;

// Number uniquely denoting the location of a sensor
typedef unsigned int Location;

class TemperatureSensor {
Public:
    TemperatureSensor(Location);
    ~TemperatureSensor() ;

    void calibrate(Temperature actualTemperature);

    Temperature currentTemperature() const;
private:
    ...
};
```

1.2 对象模型要素

主动式温度传感器的抽象：

```
//Temperature in degrees Fahrenheit
typedef float Temperature;
// Number uniquely denoting the location of a sensor
typedef unsigned int Location;

class ActiveTemperatureSensor {
public:
    ActiveTemperatureSensor(Location, void (*f)(Location, Temperature));
    ~ActiveTemperatureSensor();

    void calibrate(Temperature actualTemperature);
    void establishSetpoint(Temperature setpoint, Temperature delta);

    Temperature currentTemperature() const;

private:
    ...
};
```

培育计划对象的抽象

对于每种作物，必须有一份培育计划，描述温度、光照、营养液和其他条件应该如何随时间的变化而变化。培育计划是一种合理的实体抽象，因为它是问题域词汇的一部分。每种作物都有自己的培育计划，但是所有作物的培育计划的形式都是一样。

培育计划负责记录与培育作物有关的所有令人感兴趣的动作，对应到什么时间应采取什么动作。例如，在某种作物生长的第15天，培育计划可能为16小时内温度维持32度，开灯14小时，其他时间内温度24度。也可以希望在这天中添加某种额外的营养液，并保持微酸的PH值。

这个例子可以看出，没有对象是孤立的，每个对象都与其他对象协作，实现某些细节。

抽象：培育计划

重要特征：
名称

责任：
建立计划
修改计划
清除计划

1.2.2 封装的意义

抽象和封装是互补的概念，**抽象关注对象可以被外部观察到的行为，而封装关注这种行为的内部实现**。封装通常是通过信息隐藏来实现的（不只是数据隐藏）。信息隐藏是将那些不涉及对象本质特征的秘密都隐藏起来的过程。通常，对象的结构是隐藏的，方法的实现也是隐藏的。

抽象“帮助人们思考他们做什么”，而封装“让程序可以借助于最少的工作进行可靠的修改”。

封装在不同的抽象之间提供了明确的边界，因此导致了清晰地分离关注。例如：要从最高的抽象理解植物光合作用的工作原理，我们可以忽略一些细节，如根的责任或细胞壁的化学作用。

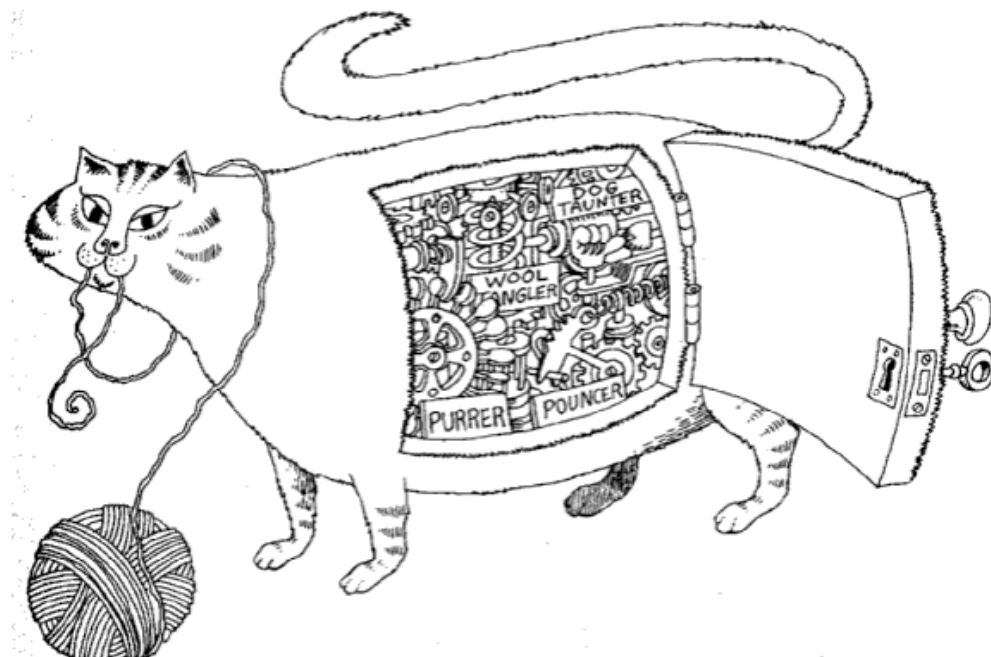
“要让**抽象**能工作，必须将**实现**封装起来”。在实践中，这意味着每个类必须有两个部分：**一个接口和一个实现**。

- ◆ 类的接口描述了它的外部视图，包含了这个类所有实例的共同行为的抽象。
- ◆ 类的实现包括抽象的表示以及实现期望的行为的方法。

通过类的接口，我们能知道客户可以对这个类的所有实例做出那些假定，类的实现封装了细节，客户不能对这些细节作出任何假定。

1.2.2 封装的意义

封装的定义：封装是一个过程，它分隔构成抽象的结构和行为的元素，封装的作用是分离抽象的概念接口及其实现。



Encapsulation hides the details of the implementation of an object.

封装隐藏了对象实现的细节

封装的例子：

回到溶液栽培的种植园系统，这个问题域中另一个关键的抽象是加热器。加热器是一种相当底层的抽象，因此我们可以决定，对这个对象执行的有意义的操作只用3种：打开它，关闭它，查看它是否正常工作。

来看看加热器的内部，会看到完全不同的情况。假设系统工程师决定将控制每个温室的计算机放在温室以外的某个地方，并通过串行通信电缆将计算机连接到传感器和执行器上。加热器类的一种合理实现可能是利用一个电磁继电器来控制每个物理加热器，继电器由电缆发送的消息控制。

假定出于某种特定的原因，系统工程师决定使用内存映射I/O来代替串行通信电缆。我们不需要改变加热器的接口，但是实现会很不一样。客户根本不会看到任何变化，因为客户只能看到加热器的接口。这就是封装的要点。

抽象：加热器

重要特征：

位置
状态

责任：

打开
关闭
提供状态信息

1.2.3 模块化的意义

“将一个程序分割到一些不同的组件中，可以在某种程度上减少它的复杂性……虽然从这一点上来说，模块化程序是有帮助的，但是**模块化程序的更大理由**是它在程序内部创造了一些定义良好的、有文档描述的边界。这些边界，或者叫接口，对理解程序是非常有价值的。”

- ◆ 如Java语言中有包的概念，包中包含类。
- ◆ 在Visual C++中，有动态链接库的概念，内部包含类和对象，被其他主程序或动态库使用。

在这些语言中，类和对象构成了系统的逻辑结构，把这些抽象放入模块中，形成系统的物理架构。特别是对一些较大型的应用来说，可能有成百上千个类，使用模块对管理复杂性有很大提高。

1.2.3 模块化的意义

模块之间的联系：大多数语言中将模块作为一个独立的概念，它们之间也区分模块的接口和它的实现。因此可以说模块化和封装是密不可分的。

模块作为一种物理容器，我们在其中声明逻辑设计中的类和对象。这和电子工程师在设计计算机主板时的情况没什么区别。NAND、NOR等逻辑门可以用来构造必要的逻辑，但是这些门必须用标准集成电路的方式进行物理封装。

在传统的结构化设计中，模块化主要是考虑对子程序进行有意义的分组，利用耦合和内聚的判据。在面向对象中，稍有不同，我们的任务是决定类和对象的物理打包，这和子程序明显不同。

1.2.3 模块化的意义



Modularity packages abstractions into discrete units.

模块化将抽象打包成独立的单元

1.2.3 模块化的意义

“分解为模块的总体目标是通过允许模块独立地设计和修改，从而减少软件的成本……每个模块的结构都应该足够简单，这样它就能被完全理解。应该能够在不知道其他模块的实现方法，并不会影响其他模块的行为的情况下，修改某个模块的实现。修改设计的容易程度应该能够满足需求变更的可能性。”因而合理的模块化可以避免巨大的、大爆炸式的重新编译发生。

开发者必须平衡两种竞争的技术考虑：封装抽象的愿望以及让其他模块看到某些抽象的需要。“有可能独立变化的细节应该成为独立模块的秘密，模块之间存在的假定只能是那些不太可能变化的东西。每个数据结构对于一个模块来说都是私有的，它可能被这个模块中的一个或几个程序访问，但模块外的程序不能访问它。任何其他程序，如果需要保存在一个的数据结构中的信息，只能通过调用这个模块的程序获得。”

1.2.3 模块化的意义

模块化就是努力创造出高内聚（将逻辑关联的抽象放在一起）、低耦合（减少模块之间的依赖关系）的模块。从这个角度出发，可以定义模块化如下：

“模块化是一个系统的属性，这个系统被分解为一组高内聚、低耦合的模块。”

因此，抽象、封装和模块化的原则是相辅相成的。**一个对象通过抽象来定义一个明确的边界（即接口），封装和模块化都围绕这种抽象提供了屏障。**

模块化的关键是发现正确的类和对象，然后把它们放到不同的模块中，这基本上是独立的设计决定。类和对象的确定是**系统逻辑设计**的一部分，而模块的确定是**系统物理设计**的一部分。我们不能在物理设计之前完成所有逻辑设计，反之亦然。设计决策是以一种迭代的方式进行的。

模块化的例子

让我们来看溶液栽培的种植园系统的模块化，操作者可以创建新的培育计划，修改老的培育计划，跟踪当前执行的培育计划。由于这里的关键抽象是培育计划，所以可能创建一个模块，目的是将所有与单个培育计划有关的类放在一起（如水果培育计划、谷物培育计划等）。这些培育计划的实现将包含在这个模块中实现。我们可能还会定义一个模块，将所有与用户界面功能相关代码放在一起。

我们设计可能还包含其他许多模块，但必须定义某个主程序，通过它来调用这些应用。**在面向对象设计中，定义这个主程序常常是最不重要的设计决策；而在传统的结构化设计中，这个主程序是根，是把所有东西聚在一起的基石。**因而面向对象的方式更为自然，将实际软件系统描述为一组服务，这一点类似于自然界或社会中系统的构成。

1.2.4 层次结构的意义

所有贴近实际可用系统中，都包含许多不同的抽象，我们不能够一下子就理解它们。一般通过封装隐藏抽象的内部视图，但还不够。一组抽象常常构成一个层次结构，通过在设计中确定这些层次结构，可以极大的简化对系统的理解，因而可以将层次结构定义为：“层次结构是抽象的一种分级或排序。”

在复杂系统中，最重要的两种层次结构是它的**类结构**（“**是一种**”层次结构）和**对象结构**（“**组成部分**”层次结构）。



抽象构成了一个层次结构

层次结构的例子1：单继承

继承是最重要的“是一种”层次结构，它是面向对象系统的基本要素。继承基本上定义了类之间的关系，在这种关系中，一个类共享了一个或多个类（分别对应于单继承和多继承）中定义的结构和行为。继承因此代表了一种抽象的层次结构，在这个层次结构中，一个子类从一个或多个父类中继承。一般来说，子类会扩展或重新定义父类中的结构和行为。

从语义上说，继承表明了“是一种”关系。例如，熊“是一种”哺乳动物，房屋“是一种”不动产，快速排序“是一种”具体的排序算法。继承因此实现了一种“一般/具体”的层次结构，其中子类将父类的一般结构和行为具体化，实际上，这也是继承的判断依据：如果B不是一种A，那么B就不应该从A中继承。

层次结构的例子1：单继承

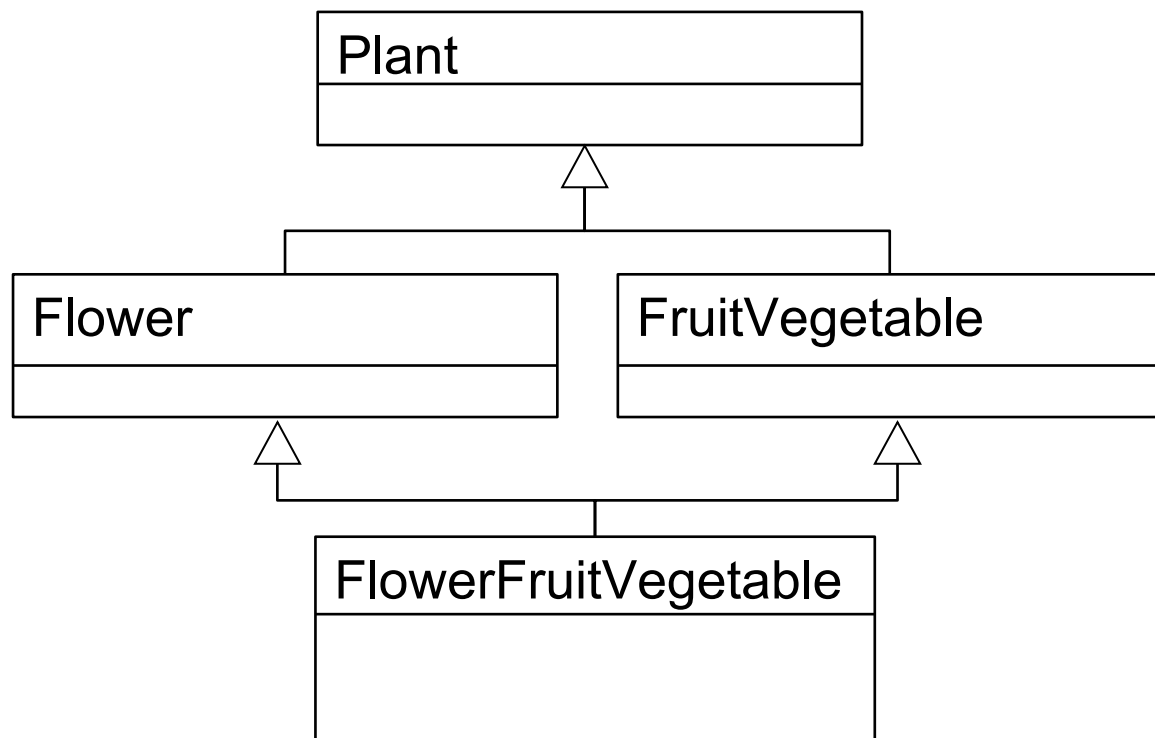
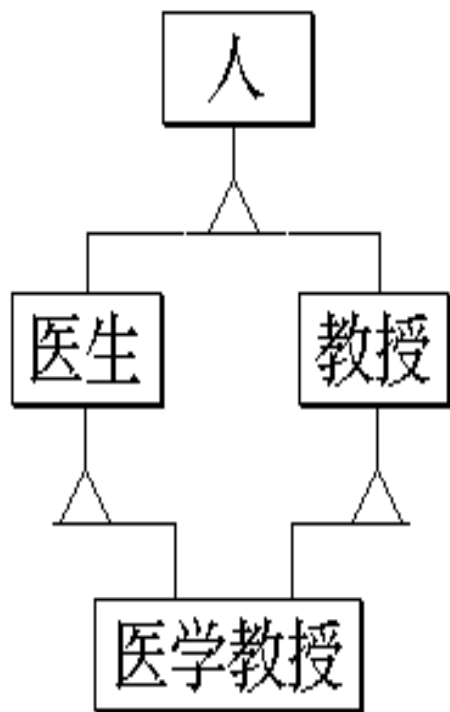
让我们来看溶液栽培的种植园系统中不同类型的培育计划。前面我们描述了非常一般化的培育计划的抽象。然而不同类型的作物需要特殊的培育计划，如水果培育计划“是一种”培育计划，谷物培育计划“是一种”培育计划，蔬菜培育计划“是一种”培育计划。这里培育计划是父类，其他是特殊化子类。

当发展继承层次结构时，不同类中共同的结构和行为会被迁移到共同的父类中。这就是常把继承称为一般/特殊层次结构的原因。

忽略“是一种”层次结构会导致膨胀的、不优雅的设计，没有继承，每个类都会是一个独立的单元，每个都要从头开发。

层次结构的例子2：多继承

对于所有水果和蔬菜，采摘的时间是抽象中的重要部分，为体现我们的设计决策，设计两个类-一个花卉（Flower）类和一个水果蔬菜（FruitVegetable）类，它们都是植物（Plant）的子类。但如果对一种既开花又结果的植物进行建模（如樱桃、李子等），对这种抽象需要设计第三个类，即花卉水果蔬菜（ FlowerFruitVegetable ）类，它同时继承花卉类和水果蔬菜类。



层次结构的例子3：聚合

“是一种”层次结构说明了一般/特殊关系，而“组成部分”层次结构则描述了聚合关系。考虑种植园的抽象，可以认为种植园包含了一些植物和一份培育计划，换言之，植物是种植园的“组成部分”，培养计划也是种植园的“培育计划”，这种“组成部分”关系被称为聚合（aggregation）。

聚合并不是面向对象的特有概念，但当聚合和继承结合使用时功能强大：聚合允许对逻辑结构进行物理分组，而继承允许这些共同的部分在不同的抽象中被复用。

聚合提出了所有权的问题，例如汽车对象中，引擎、轮胎、车体等都是汽车的组成部分，就是一种聚合关系；飞机对象中，包含机身、机翼、发动机、液压系统、航电系统等。



1.2.5 类型的意义

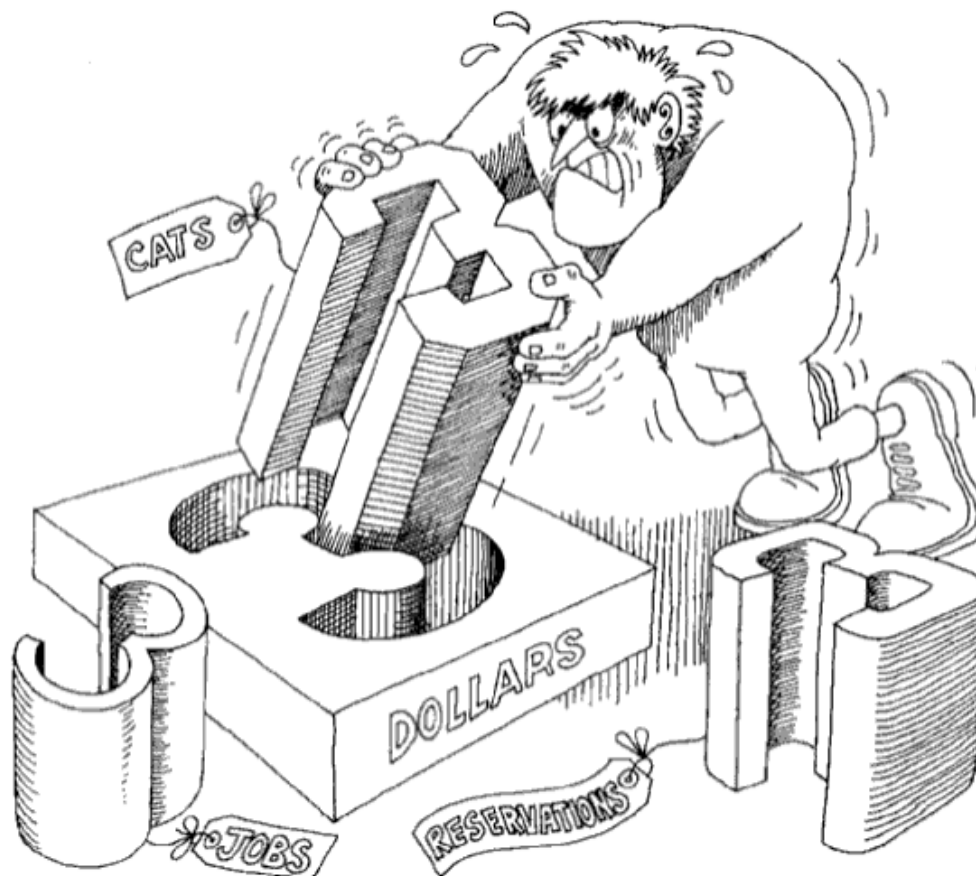
类型的概念来源于抽象数据类型的理论。Deutsch指出：“一个类型是关于结构或行为属性的准确描述，一组实体共享这些属性”。为方便起见，通常互换的使用术语**类型（type）**和**类（class）**，虽然它们概念类似，仍然把类型作为对象模型的一个独立要素，因为类型的概念对于抽象的含义有着特别不同的强调。可以将类型定义为：

“类型是关于一个对象的类的强制规定，这样一来，不同类型的对象不能够互换使用，或者至少它们的互换使用受到非常严格的限制。”

某种编程语言可以是**强类型**、**弱类型甚至无类型的**，但都可以被称为面向对象的。

类型匹配的概念是类型概念的核心。

1.2.5 类型的意义



Strong typing prevents mixing abstractions.

强类型防止将抽象搞混

1.2.5 类型的意义

在编程过程中，如何保证类型使用的安全？

(1) 使用一个类型安全的容器类，它只存储某一个类的对象，这种方法解决了不同类型的对象被错误地混用的问题。

(2) 使用某种运行时刻的类型标识，即知道当前处理的是哪一种类型的对象。利用多态可以缓解运行时对类型标识的需要。

Tesler指出使用强类型语言的一些重要好处：

- 如果没有类型检查，大部分语言的程序可能在运行时以神秘的方式“崩溃”；
- 在大多数系统中，编辑-编译-调试循环相当繁琐，所以早期的错误检查必不可少；
- 类型声明有助于为程序编写文档；
- 如果声明类型，大部分编译器可以生成更有效率的目标代码。

类型的例子：静态类型和动态类型

类型的强与弱和类型的静态与动态的概念是完全不同的。类型的强与弱指的是类型的一致性，而类型的静态与动态指的是名字与类型绑定的时间。

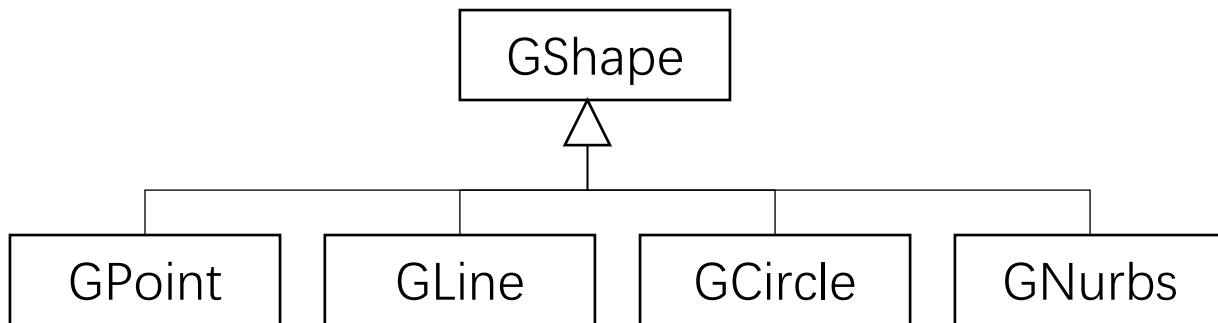
静态类型（也称静态绑定）意味着所有变量和表达式的类型在编译时绑定，

动态类型（也称动态绑定）意味着所有变量和表达式的类型直到运行时刻才知道。

例：强类型且静态类型的语言如Ada，强类型也支持动态类型的语言如C++，既无类型也支持动态类型的语言如Smalltalk。

类型的例子：多态

多态（polymorphism）是**动态类型和继承**互相作用时出现的一种情况。多态代表了类型理论中的一个概念，即一个名字（如一个变量声明）可以代表继承自同一父类的许多不同子类的对象，它所代表的对象因此可以相应**一组共同的操作**（父类的操作，但**调用对象所属子类中关于操作的实现**）。多态是面向对象语言中除了对抽象支持以外最强大的功能，区分了面向对象编程与传统抽象数据类型编程。



```
std::list<GPoint*> m_pntGroup;
std::list<GLine*> m_lineGroup;
std::list<GCircle*> m_circleGroup;
std::list<GNurbs*> m_nurbsGroup;
```

```
std::list<GShape*> m_shapeGroup;
```

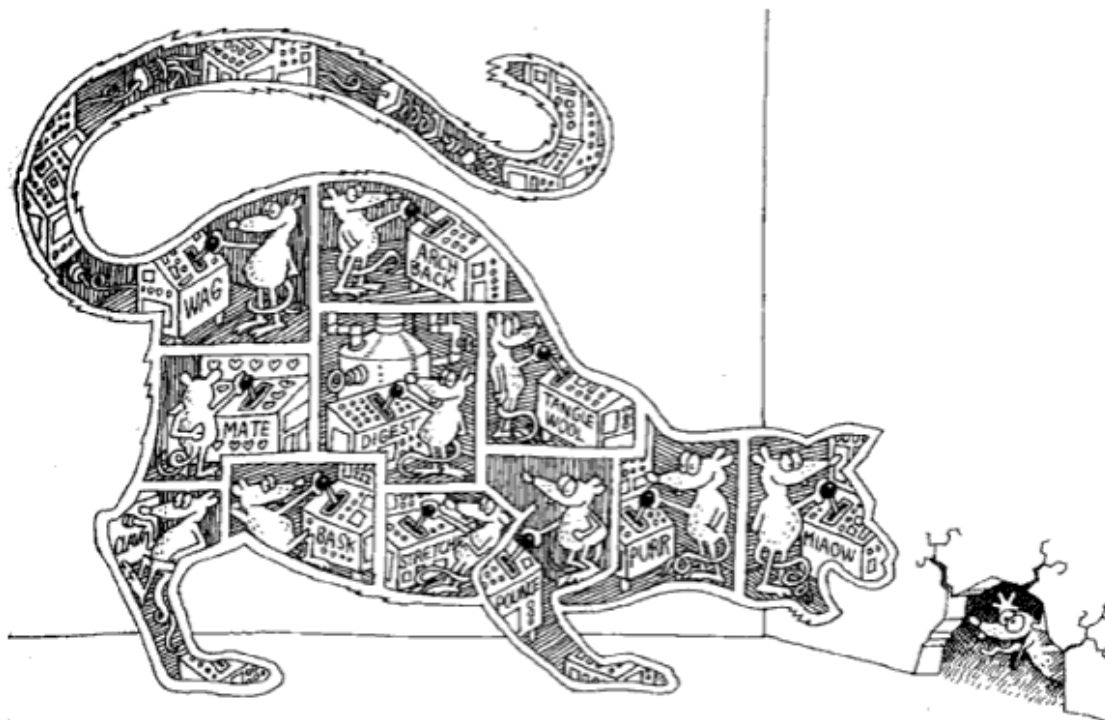
1.2.6 并发的意义

一个自动化的系统可能同时处理许多不同的事件，其中一些问题可能需要许多计算，超出了单个处理器的能力，在这种情况下，自然考虑使用一组分布式计算机来实现任务或利用多任务方式，这就是一种并发。

每个程序至少都有一个控制线程，但涉及并发的系统可能有许多线程，某些线程是短暂出现的，而另一些线程会伴随系统执行的整个生命周期。

每个对象（来自于真实世界的一个抽象）都可以代表一个独立的控制线程（一种过程抽象），这样的对象被称为“主动的”。在基于面向对象设计中，我们将世界概念化为一组协作的对象，其中某些是“主动的”。因此我们将并发定义为：“并发是一种属性，它区分主动对象和非主动对象。”

1.2.6 并发



Concurrency allows different objects to act at the same time.

并发允许不同的对象同时执行

并发的例子：

考虑一下前面提到的ActiveTemperatureSensor的传感器，它的行为要求定期测量当时的温度，然后与设定值比对，通知客户温度是否改变了一定的数值。我们不考虑这个类是如何实现这一行为的，但很清楚，需要某种形式的并发。

在实现并发时，必须注意：当一个系统引入并发时，必须考虑主动对象之间、主动对象与纯粹串行执行的对象之间，如何同步它们的活动，考虑死锁、活锁、互斥和竞争条件等问题。例如，如果两个主动对象试图给第三个对象发送消息，必须确保使用某种互斥手段，保证被调用对象的状态不会因为两个主动对象试图同时更新它而被破坏。

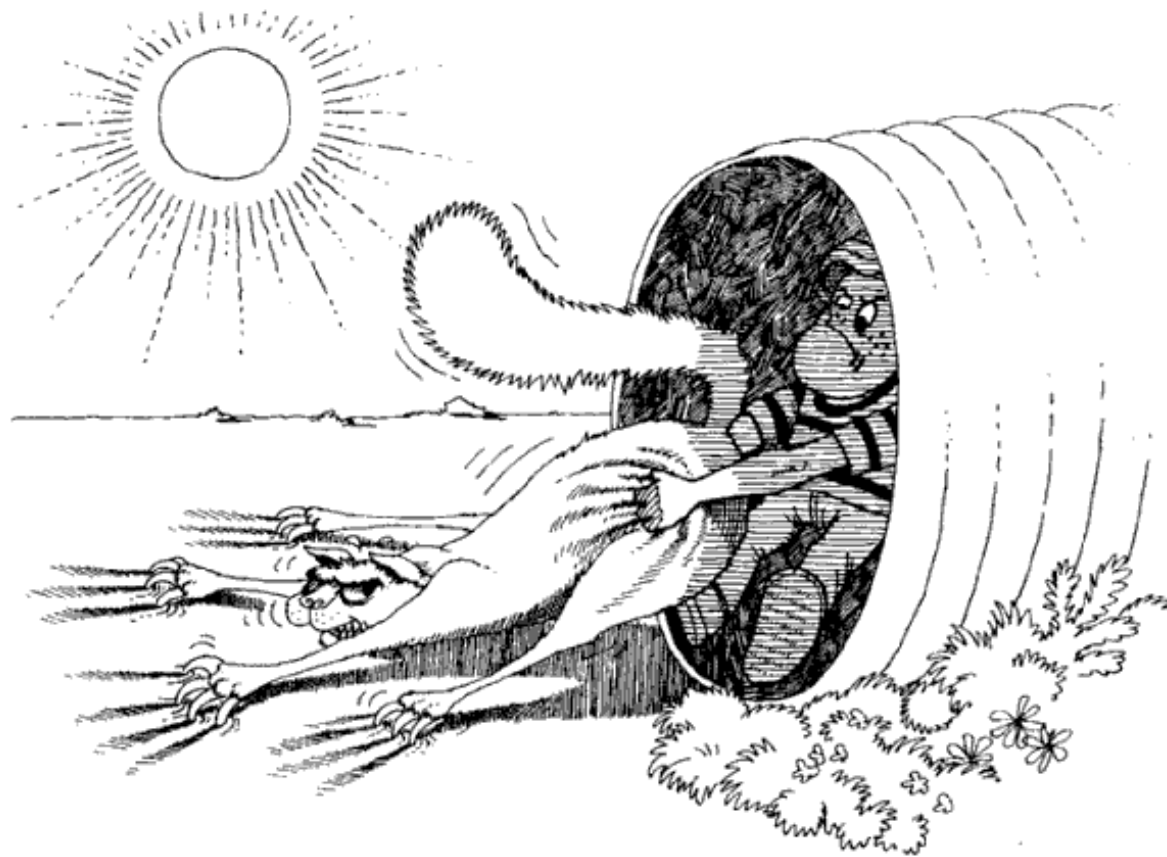
1.2.7 持久的意义

软件中一个对象会占用一定量的空间，并在一定的时间内存在。Atkinson等人指出，对象的存在时间有一个区间----从计算对象表达式时产生的瞬时对象，到保存在数据库中、超过程序执行时间的对象。

统一对象和并发的概念导致了并发的面向对象程序设计语言。与此类似，在对象模型中引入持久的概念导致了面向对象的数据库。在实践中，这样的数据库建立在已证明的技术之上，如序列化、排序、层次数据库、网状数据库或关系数据库模型。但是，它为程序员提供了一种面向对象接口的抽象，数据库查询和其他操作都是通过这个接口，以对象的方式完成，这些对象的生命周期超过了单个程序的生命周期。这种统一极大地简化了某些类型应用的开发。具体来时，它允许我们对数据库和应用的非数据部分使用同样的设计方法。

综上所述，我们可以这样定义持久：“持久是对象的一种属性，利用这种属性，对象跨越时间（例如，当对象的创建不存在时，对象仍然存在）和空间（例如，对象的位置从它被创建的地址空间移开，存入数据库）而存在。”

1.2.7 持久的意义



Persistence saves the state and class of an object across time or space.

持久跨越时间或空间保存了一个对象的状态和类型

通过对对象模型的分析可以看到，对象模型从本质上与传统的结构化分析、结构化设计和结构化编程方法所使用的模型是很不一样，但这并不是说对象模型放弃了较老方法中好的原则和经验，相反，它引入了新的元素，建立在这些更早的模型之上。

对象模型的好处：

- 使用对象模型帮助我们探索基于面向对象和面向对象编程语言的表达能力；
- 利用对象模型不仅鼓励软件的复用，而且鼓励整个设计的复用，这导致了可复用应用框架的产生；
- 使用对象模型将得到构建在稳定的中间状态之上的系统，这样的系统更适合变化。这意味着，这样的系统可以随时间而进化，而不是在对需求进行第一次主要修改时就抛弃原来的系统或者完全重新设计。



02

类与对象



- ◆ 2.1 理解类与对象
- ◆ 2.2 对象的本质
- ◆ 2.3 对象之间的关系
- ◆ 2.4 类的本质
- ◆ 2.5 类之间的关系
- ◆ 2.6 创建高品质的类与对象

类与对象

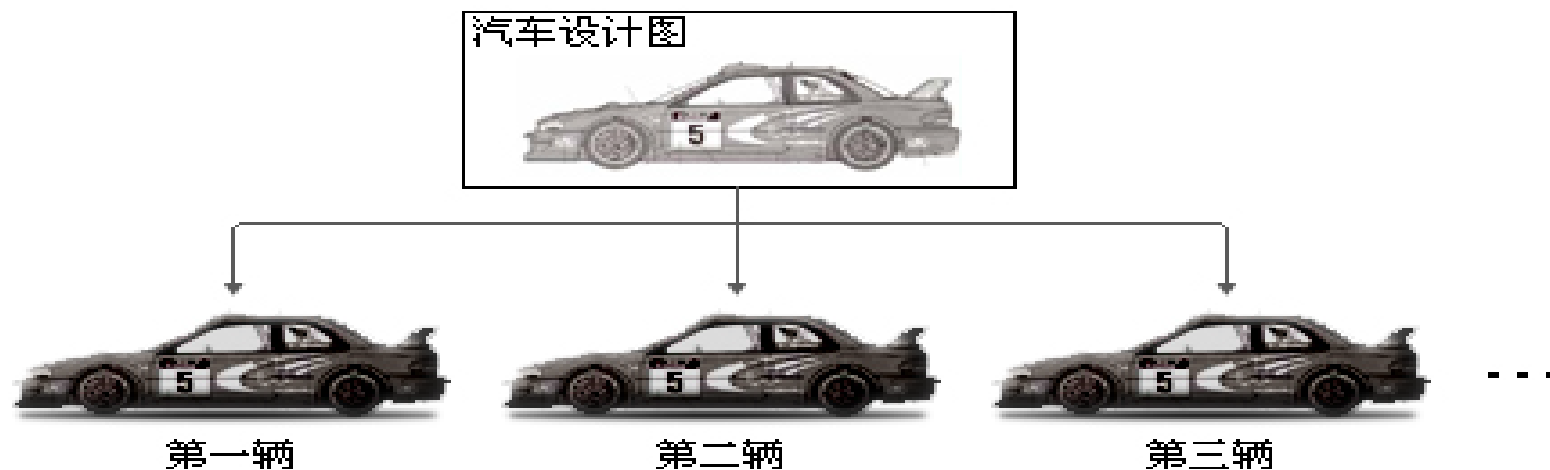
考虑：什么是眼镜。给眼镜下定义。

大脑中对眼镜的概念就是对眼镜这类事物抽象出来的一个类，而这个类在现实中的存在就是对象。

类在这里可以理解成种类，现实世界中，存在很多种类的对象，桌子、书、自行车。

其实类可以称为对象的一个模板。

类是对某一类事物的描述，是抽象的、概念上的定义；对象是实际存在的该类事物的每个个体，因而也称实例(instance)。



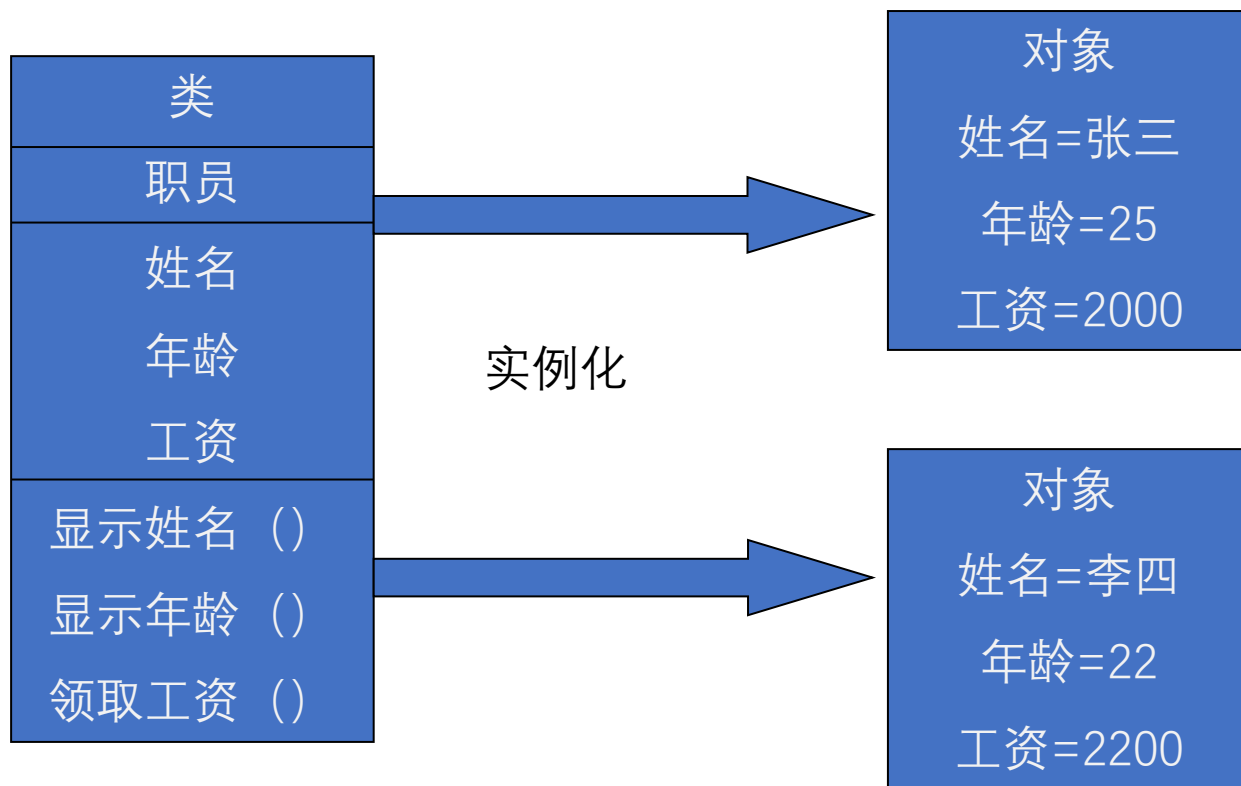
汽车设计图为例，它包括了对要设计汽车的全部描述和方法。那么按照这个设计图纸所生产的汽车则为在该类下所产生的对象。

类是对象的描述，对象是类的实例。

注意：一个类可以产生多个对象，并且初始状态都一样，其中一个对象改变后，不会影响类；但类改变则会影响所有由它产生的对象。

靠什么
保证？

例子：



2.1 类与对象

对象



属性

型号

价格

里程

行为

行驶

起动

停车



属性

车轮数量

档的数量

行为

刹车

加速

换档



属性

名称

颜色

品种

行为

犬 吠

摇尾巴

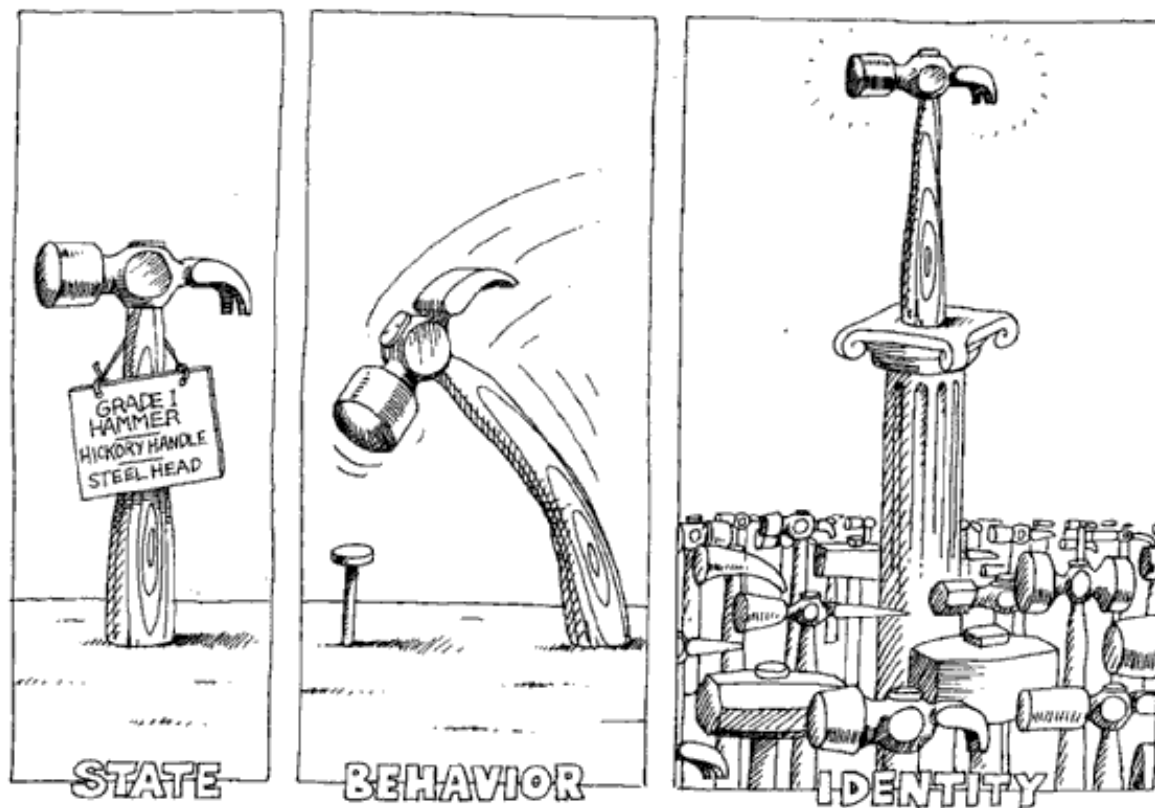
吃东西

可以非正式地将对象定义为一个可以触摸的实体，展示一些定义良好的行为。从人类认知的角度看，对象可以是下列事物之一：

- 一个可以触摸或可以看见的东西；
- 在智力上可以理解的东西；
- 可以指导思考或行动的东西。

在软件中，“对象”这个术语首先正式出现在Simula语言中，在于Simula程序中对象通常用于模拟真实世界的某个方面。

Grady Booch如下定义对象：一个对象是一个具有状态、行为和标识符的实体。结构和行为类似的对象定义在它们共同的类中。“实例”和“对象”这两个术语可以互换使用。



An object has state, exhibits some well-defined behavior, and has a unique identity.

一个对象具有状态，能展示某种定义良好的行为，且具有唯一的标识符

2.2.1 状态

考虑一个分发饮料的自动售货机。这类对象的一般行为是当某人在投币口塞进钱，并按下选择按钮时，机器就会提供一种饮料。如果用户先选择饮料再向投币口塞钱会怎样？大部分自动售货机什么也不会干，因为用户违反了他们基本的操作假定。换言之，用户没有注意到（先进行了选择）自动售货机处于一种状态（等待塞钱）。

在上述情况下，可以看到对象的行为受到它操作顺序的影响，这种行为依赖事件或依赖时间的原因是对象内部存在状态。例如：自动售货机的一个基本状态是等待用户塞入一定数量的钱，然后再进行饮料类型和数量的选择。

从这个例子中，我们可以得到下面的基本定义：对象的状态包括这个对象的所有属性（通常是静态的）以及每个属性当前的值（通常是动态的）。

自动售货机的另一个属性是它可以接受钱币，这是一个静态属性，与之相对，某一时刻它接受的金额代表了这个属性的动态值，它受到对机器操作次序的影响，当用户塞入钱币时，数量增加，当提供饮料后，数量减少。

每个对象在生成后，都存在一个构造函数所赋予的状态，在使用过程中，属性值不断变化，直到对象被析构函数销毁。

2.2.2 行为

没有对象是孤立存在的。对象与对象之间会相互操作。因此，可以这样定义：
“行为是对象在状态改变时的动作和消息传递时的反应方式。”换言之，对象的行为代表了它外部可见的活动（或操作）。

一个操作是某种动作，一个对象对另一个对象执行这个操作，目的是获得反应。例如，客户可能调用append和pop操作，分别使一个队列对象增长或缩减。

一般情况下，一个对象调用另一个对象的成员函数，也可以说成“一个对象向另一个对象传递一个消息”。这里互换使用了消息和操作两个术语。

消息传递只是定义对象行为的一个方面，对象行为还会受到其状态的影响。考虑自动售货机的例子，我们可以调用某个操作选择商品，但自动售货机的行为取决于它自身的状态，如果没有塞进足够购买所选商品的钱，自动售货机可能什么都不会做。如果提供了足够的零钱，自动售货机会收下钱，然后给出所选商品（然后改变自身状态）。因而可以说：一个对象的行为是它的状态以及施加在它上面的操作的函数。

状态则可以定义为：一个对象的状态代表了它的行为的累积。

(1) 操作

一个操作代表了一个类提供给它的对象的一种服务。在应用中，队列类的一个客户通常执行一个对象的5种操作：

- 修改操作 更改一个对象状态的操作；
- 选择操作 访问一个对象的状态但并不更改这个状态的操作；
- 遍历操作 以一种定义良好的方式访问一个对象的所有部分的操作
- 构造操作 创建一个对象并初始化它的状态的操作；
- 析构操作 释放一个对象的状态并销毁对象本身的操作

(2) 角色和责任

对象在它们的生命周期中扮演不同的角色，这些角色往往是动态的，同时又是互斥的。

一个对象可提供多种操作，可通过分组划分它的操作空间，这表明一个对象可以扮演角色。角色是一个对象戴上的一个“面具”，它定义了一种抽象与它的客户之间的契约，它的责任是它为支持所有契约提供的全部服务。

实际上，很多对象在它们的生命周期中需要扮演许多不同的角色，考虑下面的例子：

- ◆ 一个银行账户可能扮演一个现金资产的角色，账户所有者向它存钱或者从它取钱。但是，对于税务检查来说，这个账户可能扮演这样一个角色，即每年都必须报告它的股息；
- ◆ 在一天中，同一个人可能扮演母亲、医生、园丁和电影评论员的角色。

2.2.3 标识符

标识符是一个对象的属性，它区分这个对象和其他所有对象。大多数程序设计语言使用变量名称来区分临时对象，混淆了定址能力和标识符。

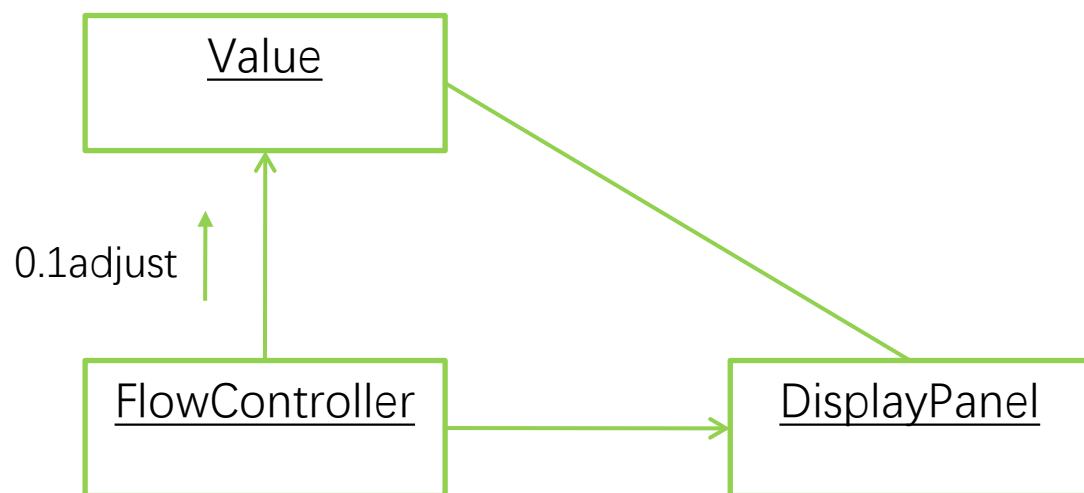
每个对象的唯一标识符（不一定是名称）是在对象的整个生命周期中都被保持的，即使它的状态改变时也是如此。



由于该对象既没有直接名称也没有间接名称，它的标识符被遗失，造成内存泄露。

链接(link)关系:

链接关系指两个对象在物理上或概念上的联系，一个对象通过它与其他对象的关联，实现与其他对象的协作。通过这种关联，一个对象（客户）可以请求另一个对象（服务提供者）的服务，也可从一个对象导航到另一个对象。



水流控制系统例子:

FlowController对象调用Value对象上的操作（目的是改变它的设置）和DisplayPanel对象上的操作（目的是改变它显示的内容）。这种分离关注在结构良好的面向对象系统中很常见。

作为链接的参与者，对象可能扮演以下三种角色之一：

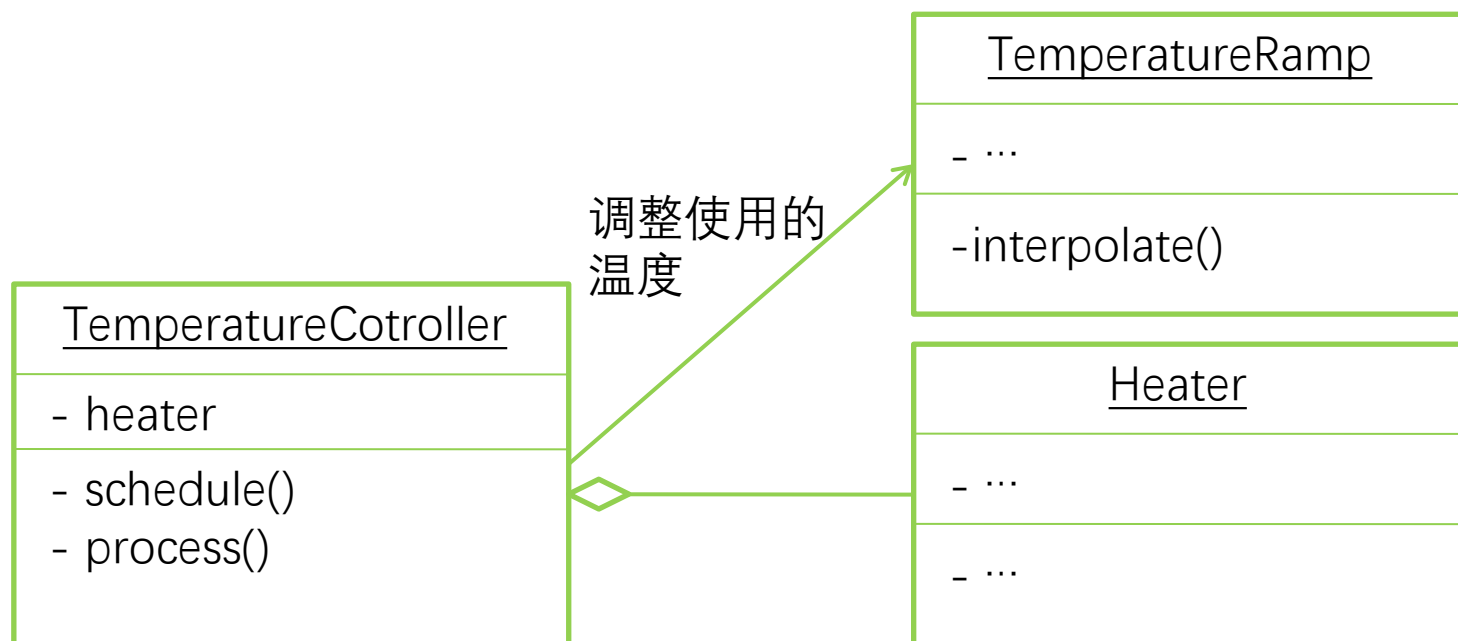
- ❑ **控制器**：这个对象可以操作其他对象，但不会被其他对象操作。
- ❑ **服务器**：这个对象不操作其他对象，它只被其他对象操作。
- ❑ **代理**：这个对象既可以操作其他对象，也可以被其他对象操作。

聚合关系：

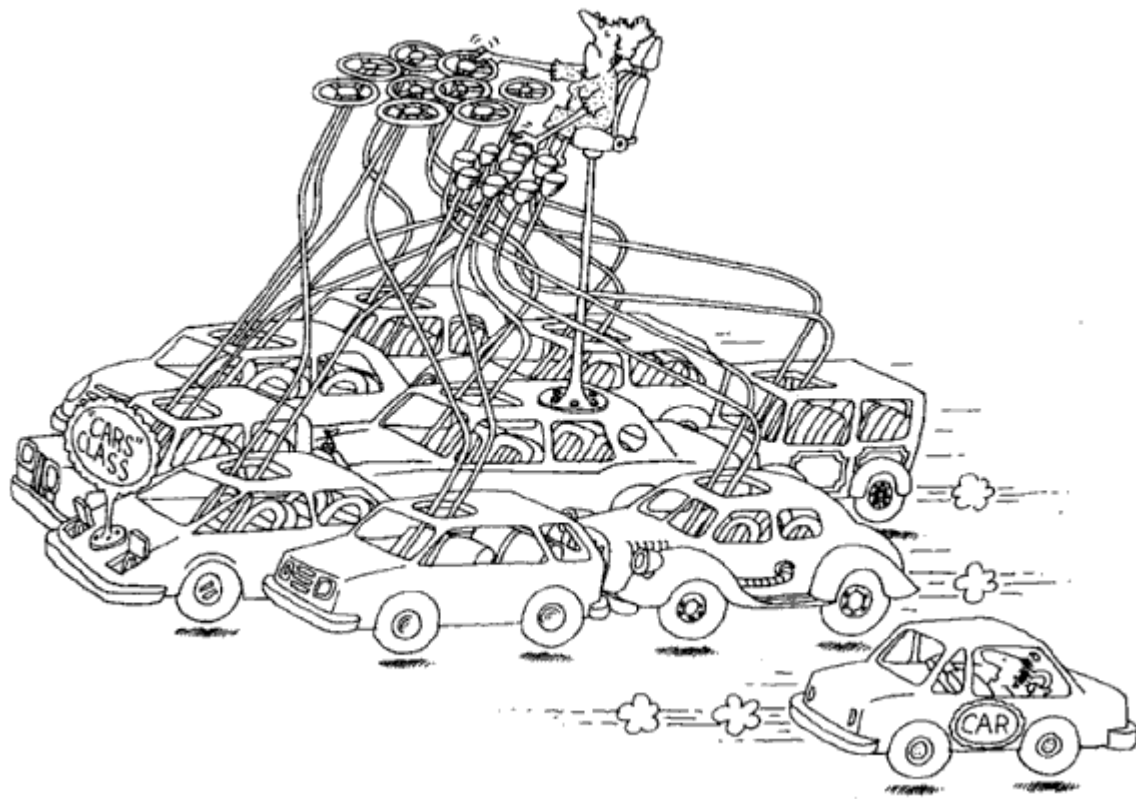
聚合关系表明了一种**整体/部分层次结构**，提供了从整体（也称聚合体）导航到它的部分的能力。聚合可以代表物理上的包含，也可以不是。

例如：一架飞机由机翼、引擎、起落架、航电系统等组成，这是物理上的包含关系。股票持有人拥有股票，但这些股票不是股票持有人的物理组成部分。

温度控制器的例子：



对象是存在于时间和空间中的具体实体，而类仅代表一种抽象，即一类对象的“本质”。因而可以将类定义为：类是一组对象，它们拥有共同的结构、共同的行为和共同的语义。”一个对象就是一个类的实例。



A class represents a set of objects that share a common structure and a common behavior.

接口和实现

编程在很大程度上是一种 **“制定契约”**：一个大型问题的不同功能通过子契约被分配给不同的设计元素，被分解成较小的问题。设计类时更能体现这一思想。

一个单独的对象是一个具体实体，在整体系统中扮演某个角色，而类则记录了所有相关对象的共同结构和行为。因此，类起到的作用是在一种抽象和所有它的客户之间建立协议，即类的接口。

编程即契约的观点可以区分一个类的外部视图和内部视图。一个类的接口提供它的外部视图，因此强调了抽象，隐藏了它的结构和行为的秘密。接口由所有的操作**声明**构成。类的实现是它的内部视图，包含类行为的秘密。一个类的实现主要由类接口中定义的所有操作的实现构成。

类的接口分4类：**公有、保护、私有和包**（只有同一个包中的类才能访问的声明）。

考虑这些对象的类之间的相似和不同-花、雏菊、红玫瑰、黄玫瑰、花瓣和瓢虫，我们可以得到下列结论：

- ◆ 雏菊是一种花；
- ◆ 玫瑰是另一种花；
- ◆ 红玫瑰和黄玫瑰都是一种玫瑰；
- ◆ 花瓣是两种花的组成部分；
- ◆ 瓢虫会吃掉蚜虫等害虫，这些害虫会侵扰某些种类的花。

从这个简单的例子，我们可以知道类也像对象一样，不是孤立存在的。来看一下这些类之间的关系，首先，一种类关系可能表明某种类型的共享，例如雏菊和玫瑰都是花，这意味着它们都有花瓣，散发着芬芳。其次，一种关系可能表组成关系，花由花瓣组成，还有一种关系是：瓢虫和花之间存在一种共生关系，瓢虫保护花免遭害虫侵害，花为瓢虫提供食物来源。



总结来看，一般类之间存在4中基本类型的关系：

继承关系：一般/特殊关系，表示“是一种”关系。例如：玫瑰是一种花，这意味着玫瑰类是花类的子类。

聚合关系：整体/部分关系，表示“组成部分”关系。例如：花瓣不是一种花，它是花的一个组成部分。

关联关系：一种语义上的依赖关系，如果没有这层关系，这些类就毫无关联了。如瓢虫和花之间的共生关系，瓢虫保护花免遭害虫侵害，花为瓢虫提供食物来源。

依赖关系：处于这种关系一端的元素以某种方式依赖于处于另一端的元素。这警告设计者，如果其中一个元素发生了改变，可能会影响到另一种元素。

2026

THANK YOU

浙江大学机械工程学院航空制造工程研究所
王青 李江雄